

Tutorial 5(21/8/26)

1. A bank wants to develop a system to manage customer accounts. Each account has an account number, account holder name, and balance. The balance should not be modified directly. Customers should be able to deposit and withdraw money.

- Create a BankAccount class.
- Apply **encapsulation** to protect the balance.
- Use a constructor to initialize account details.
- Implement deposit() and withdraw() methods.
- Display the updated account details.

2. A university has different types of employees such as Professor, LabAssistant, and AdministrativeStaff. All employees have a name, employee ID, and basic salary, but salary calculation differs for each employee type.

- Create a parent Employee class.
- Create appropriate child classes.
- **Override** a calculateSalary() method.
- Display the salary of different employees.

3. An e-commerce application supports payment through **UPI, Credit Card, and Net Banking**. Every payment method must provide a makePayment() operation, but the implementation is different for each method.

- Create a suitable abstract class or interface.
- Implement different payment methods.
- Use polymorphism to process payments.
- Allow the user to select a payment method.

4. An online shopping application sells **Electronics, Clothing, and Books**. Every product has a product ID, name, and price. Each category provides a different discount.

- Create an abstract Product class.
- Create subclasses for Electronics, Clothing, and Books.
- Implement calculateDiscount().
- Use polymorphism to calculate the final price.

5. A graphics application needs to calculate the area of different shapes such as circles, rectangles, and triangles.

- Create an abstract Shape class.
- Define an abstract calculateArea() method.
- Create subclasses for each shape.
- Accept dimensions from the user.
- Display the area using polymorphism.

6. A company is conducting a campus placement drive. Each candidate is assigned a unique **Candidate ID** and receives scores in three assessment categories:

- **Aptitude**
- **Technical**
- **Communication**

The placement team wants to shortlist the **Top K candidates** based on their overall score. The overall score of a candidate is calculated as:

Overall Score = Aptitude + Technical + Communication

If two or more candidates have the same overall score, the candidate with the **smaller Candidate ID** should be given higher priority. Write a Java program to identify and display the **Top K candidates**.

1. Create a class named Candidate.
2. The Candidate class should contain the following data members:

- Candidate_Id
 - name
 - aptitude
 - technical
 - communication
3. Create a parameterized constructor to initialize the candidate details.
 4. Define a method getTotalScore() to calculate and return the overall score.
 5. Use an array of Candidate objects to store all candidates.
 6. Sort/rank the candidates according to the given priority rules.
 7. Display only the Top K candidates.

Input Format:

N K

N – Number of candidates K – Number of candidates to be shortlisted

CandidateID ,Name, Aptitude, Technical, Communication

Constraints

$$1 \leq K \leq N \leq 1000$$

$$1 \leq \text{CandidateID} \leq 10^6$$

$$0 \leq \text{Aptitude} \leq 100$$

$$0 \leq \text{Technical} \leq 100$$

$$0 \leq \text{Communication} \leq 100$$

Test Case 1

Input

5 3

101 Arun 80 90 70

102 Priya 90 85 80

103 Ravi 70 95 75

104 Divya 85 85 85

105 Kiran 80 80 70

Output

102 Priya 255

104 Divya 255

101 Arun 240